

PFC AI Design Agent

Global State Schema

& Next-Level Roadmap

How all agents — calculation, documentation, simulation, database, JS tools
and all future agents — share a single source of truth without passing wrong data

Plus: a tiered roadmap of what the platform can become

Version 1.0 · June 2026 · No source code changes

This document answers two strategic questions. First: how do multiple agents share data correctly as the system grows from 4 agents today to 8 or more in the future? The answer is a Canonical Design State — a single versioned schema that every agent reads from and writes to by contract. Second: what does this platform look like at its full potential? The answer is a tiered roadmap across three time horizons.

Part 1 — The Multi-Agent Data Problem

Today the system has: a Calculation Agent (Python backend), a Documentation Agent (to be built), a Database Agent (magnetics library), and two embedded JS tools (ReviewMagnetics, ControlDesign). Tomorrow it will have a Simulation Agent, more JS tools, semiconductor and EMI calculation agents, and possibly an AI design advisor. Every one of these agents needs the same design data — but in different formats and for different purposes.

■ CAUTION The Problem: Ad-Hoc Data Passing Breaks as the System Grows

Today, Step 7 passes data to Step 15 as a loosely typed dict. Step 15 passes data to the Control Design JS tool as a `postMessage` params object. The Report Generator receives a mix of `approved_design` and `step15_result` and `step16_params`. Each agent knows its neighbours but no one has a complete picture. When a new agent is added, it must negotiate with every existing agent individually. This scales as $O(N^2)$ — each new agent adds N integration points, not 1. At 8 agents that is 28 integration points. This is how "adding one agent" turns into three weeks of debugging.

INSIGHT The Solution: One Canonical Schema That Every Agent Signs

Replace all ad-hoc data passing with one versioned `DesignState` schema. Every agent reads from it and writes to it using the same field names, the same types, and the same units. When a new agent is added, it learns the schema — not the API of every other agent. This scales as $O(N)$: each new agent adds 1 integration point, not N . At 8 agents that is 8 integration points, not 28.

1.1 Current State — $O(N^2)$ Integration Points



Each agent has a direct, undocumented contract with its neighbours. Adding the Simulation Agent requires negotiating with every agent above.

1.2 Target State — $O(N)$ with Canonical DesignState



Each agent reads/writes the canonical schema. Adding a future agent = learning one schema, not negotiating with 5 existing agents.

Part 2 — The Canonical DesignState Schema

The DesignState is a single JSON document that represents the complete state of one PFC design from first specification through final approval. It has a version field, a project metadata block, and one section per chapter. Sections are added as the designer progresses; unstarted sections are null.

✓ AGREED The DesignState is the Single Source of Truth

Every agent reads from and writes to the DesignState. No agent passes data directly to another agent. No agent holds private state that other agents cannot see. The frontend (App.tsx) holds the live DesignState in memory. The backend receives the relevant section(s) in every API request. The DesignState is versioned — schema_version = "1.0" today, "1.1" when a new field is added, "2.0" when a breaking change is made.

2.1 Top-Level Structure

Field · Type · Units · Description · Written by

Field	Type	Units	Description	Written by
schema_version	string	—	Schema version for compatibility checking. "1.0" initial.	System
project_id	string	—	Unique design identifier. Used in filenames and report titles.	Frontend
created_at	ISO datetime	UTC	When the design was first created.	Frontend
last_modified	ISO datetime	UTC	Timestamp of last approved step.	Frontend
s1_intake	IntakeState null	—	All input specifications: Vin, Vout, Pout, standards, 9-point matrix.	Mode A
s2_topology	TopologyState null	—	Selected topology, mode, controller IC, channels, fsw.	Mode A
s3_inductor_sizing	InductorSizingState null	—	L_target, currents, K(D), material, core, wire selection, first-pass losses.	Calc Agent
s4_inductor_perf	InductorPerformanceState null	—	Waveforms, 3D model, detailed losses, thermal, audit. JS studio output captured here.	Calc Agent + JS
s5_capacitor	CapacitorState null	—	Bank sizing, part selection, ripple verification, lifetime analysis.	Calc Agent
s6_control	ControlState null	—	Both loop designs, stability scorecard, BOM.	Calc Agent + JS

s7_semiconductors	SemiconductorState null	—	MOSFET, diode, rectifier, heatsink. (Future Chapter 7)	Calc Agent
s8_protection	ProtectionState null	—	MOV, relay, fuse, inrush. (Future Chapter 8)	Calc Agent
s9_emi	EMIState null	—	DM/CM filter design, insertion loss. (Future Chapter 9)	Calc Agent
sim_results	SimResults null	—	Simulation outputs captured from JS tools for inclusion in report.	Sim Agent
doc_metadata	DocMetadata null	—	Report revision history, author, approval status per chapter.	Doc Agent
js_state	JSToolsState null	—	Current state of all embedded JS tools: component values, user adjustments.	JS Tools
material_cache	MaterialCache null	—	Resolved material data from DB Agent. Avoids repeated DB calls.	DB Agent

2.2 Key InductorSizingState Fields (Example Section)

Every section of the DesignState is documented at this level of detail. Below shows s3_inductor_sizing as an example.

Field	Type	Units	Description	Written by
L_target_uH	float	μH	Target per-phase inductance from ripple specification.	Calc Agent
I_pk_A	float	A	Peak per-phase current at worst operating point.	Calc Agent
I_rms_A	float	A	RMS per-phase current at worst operating point.	Calc Agent
I_dc_crest_A	float	A	DC bias current at full-load crest (used for core selection).	Calc Agent
K_D_table	float[9]	—	Ripple cancellation factor K(D) at all 9 operating points.	Calc Agent
material_key	string	—	Selected material identifier: e.g. "edge_60". Links to material_cache.	Calc Agent
core_part_number	string	—	Selected core part number: e.g. "0059392A2".	Calc Agent
stacks	int	—	Number of stacked cores (1, 2, or 3).	Calc Agent
N_turns	int	—	Number of winding turns.	Calc Agent
wire_designation	string	—	Wire identifier: e.g. "TRW(F)(L)-0.1 × 200".	Calc Agent
FFcu_pct	float	%	Bare copper window fill factor.	Calc Agent
Ku_pct	float	%	Insulated bundle fill factor.	Calc Agent
Pcu_est_W	float	W	First-pass copper loss estimate at worst operating point.	Calc Agent
Pcore_est_W	float	W	First-pass core loss estimate (peak-point) at worst op. point.	Calc Agent

dT_est_C	float	°C	First-pass thermal estimate at worst operating point.	Calc Agent
selected_at	ISO datetime	UTC	When this section was approved by the designer.	Frontend

Part 3 — Agent Contracts: What Each Agent Reads and Writes

The schema defines not just the fields but the access rights of each agent. No agent is allowed to write to a section it did not create. This prevents silent overwriting of approved values.

Agent	Reads From	Writes To	Access Rule
Calculation Agent (Python backend)	s1_intake, s2_topology, material_cache Previous sections as inputs to current section	s3_inductor_sizing, s4_inductor_perf, s5_capacitor, s6_control, s7..s9 (future)	Writes only to its own output section. Never modifies intake or topology.
Documentation Agent (report generator)	ALL sections (read-only) All approved results	doc_metadata only (revision, approval status)	Read-only on all design data. The only write is to doc_metadata — chapter approval status.
Simulation Agent (JS tools + Python)	The section relevant to the current sim: s3/s4 for magnetics, s6 for control	sim_results js_state	Never modifies calculation results. Captures what the designer sees in the JS tool.
Database Agent (magnetics library)	DB queries (material_key, operating conditions)	material_cache only	Provides read-only library data. Never touches any design section.
JS Tools (browser iframes)	Relevant parameters via postMessage: lphi_uH, co_uF, vout_V, etc.	js_state (component values, user adjustments)	Cannot write to calculation results. User adjustments recorded in js_state.
Any Future Agent	The sections relevant to its purpose (declared in the agent's manifest)	Only new fields it introduces OR a new null-initialised section	Future agents declare their read/write manifest. Never write to sections owned by existing agents.

3.1 How a New Agent Joins Without Breaking Existing Agents

- **Step 1 — Declare intent:** The new agent's manifest lists which fields it reads and which it writes. Fields it will write must not already be owned by an existing agent.
- **Step 2 — Add new fields to schema:** If the agent needs new fields, they are added to the DesignState schema as optional (null-initialised). Existing agents ignore fields they do not know about.
- **Step 3 — Increment schema_version:** "1.0" → "1.1" for new optional fields; "1.0" → "2.0" for any breaking change (rename or remove an existing field).
- **Step 4 — All agents check version:** Each agent declares the minimum schema version it requires. Agents with lower minimums continue to work.
- **Step 5 — No existing agent changes:** Adding a new agent with new fields requires zero changes to existing calculation, documentation, or database agents.

✓ RULE The Golden Rule of Multi-Agent State

An agent may only write to fields it created. It may read from any field. If two agents both need to update the same field, one of them is wrong — redesign the ownership, not the field.

Part 4 — Practical Implementation: What Changes in the Code

Adopting the Canonical DesignState does not require a rewrite. The current App.tsx already holds a design state object. The change is to replace the ad-hoc nested object with a formally typed, documented schema.

4.1 The Three Files That Define the Schema

File	Language	Purpose	Audience
src/types/DesignState.ts	TypeScript	TypeScript interface definitions for all sections. Frontend compiles against this — type errors caught at build time.	Frontend developer Documentation Agent
backend/app/schemas.py	Python / Pydantic	Pydantic models matching DesignState.ts exactly. FastAPI validates all requests against this — wrong data caught at runtime.	Backend developer Calculation Agent
docs/DESIGN_STATE.md	Markdown	Human-readable documentation of every field: name, type, units, physical meaning, which agent writes it, example value. This is the specification all future agents are written against.	All agents All developers

4.2 How JS Tools Connect to the Canonical Schema

The postMessage bridge already exists. The improvement is to formalise what is sent. Instead of ad-hoc params objects, every JS tool injection sends a defined sub-section of the DesignState:

Current (ad-hoc):

```
postMessage({ type: "setPythonValues", params: { lphi_uH: 235, rL_m0hm: 28, co_uF: 2350, ... } })
```

Future (schema-typed):

```
postMessage({ type: "setDesignState", section: "s6_control", data: designState.s6_control // typed ControlState object })
```

The JS tool receives a well-typed, documented object. It cannot silently receive undefined fields — the schema contract prevents it.

4.3 Schema Versioning in Practice

- **"1.0" → "1.1" (additive):** New optional field added. All agents still work. New agents can use the field; old agents ignore it.
- **"1.1" → "1.2" (additive):** Another new section (s7_semiconductors) added as null. No existing agent touched.
- **"1.x" → "2.0" (breaking):** A field is renamed or removed. Rare. Requires all agents to be updated. Document the migration in DESIGN_STATE.md.

Breaking changes (2.x) should only happen during major platform redesigns, not during incremental feature additions.

Part 5 — Next-Level Roadmap: Three Tiers

The following roadmap describes what this platform can become across three time horizons. Tier 1 uses the current architecture and can begin immediately after the current implementation. Tiers 2 and 3 require new components but none of them require redesigning what already exists.

Tier 1 — Depth and Polish	<i>Months 1–3 after current implementation</i>
■ Canonical DesignState schema	Implement DesignState.ts, schemas.py, and DESIGN_STATE.md as described in Part 4. This is the enabler for everything else on the roadmap.
■ Complete documentation standards	Run the Documentation Agent on all 6 chapters — apply equation format, splash pages, structural elements, DATA SOURCE boxes, and interpretation paragraphs.
■ Tolerance and sensitivity analysis	Add AL $\pm 8\%$ tolerance columns to all inductance and loss tables. Add a Monte Carlo sensitivity run showing the distribution of ΔT across AL tolerance.
■ Three-method capacitor lifetime	Fully implement all three Arrhenius methods with the confidence band interpretation paragraph as documented in the capacitor calculation reference document.
■ Control theory Section 6.0	Write the complete 15-subsection control theory foundation. Derive $G_{id}(s)$ and $G_{vp}(s)$ from circuit equations. Show the open-loop instability Bode plot.
■ Measurement plan chapter	Add the bench measurement and validation plan at the end of Chapter 4 and Chapter 6 — what to measure, how, and what it confirms.
Tier 2 — Intelligence and Automation	<i>Months 4–9</i>
■ AI Design Advisor	An LLM-powered agent that reviews the complete approved design and asks probing questions. "Your PM at 264 Vac is 47° — with $\pm 5\%$ component tolerances the worst case could be 38° . Have you verified the design at tolerance corners?" This agent reads the DesignState and writes advisory notes to doc_metadata.
■ Design Optimisation Engine	For a given specification, compute not just a passing design but the Pareto-optimal frontier: minimum loss vs minimum size vs minimum cost. Present the designer with three recommended designs on the frontier rather than one.

■ Monte Carlo Analysis	Run the design 1000 times with component values drawn from their tolerance distributions. Show the designer: "98% of builds will have $\Delta T < 35^{\circ}\text{C}$. 2% will exceed the budget at the AL-min corner at 264 Vac." Much more useful than a single worst-case number.
■ Design History and Comparison	Store all approved designs in a database. A designer can open a previous design, compare it to the current one, and see exactly what changed and why. Full version control for designs — not just code.
■ Interactive JS Simulation Tools (expanded)	Add waveform simulators for: switching node voltage/current (MOSFET gate-drain-source), EMI filter input/output spectrum, bridge rectifier current waveform. Each tool follows the same iframe+postMessage architecture as ReviewMagnetics.
■ Datasheet Import via AI	Upload a component datasheet PDF. The Database Agent extracts key parameters (Steinmetz constants, thermal resistance, rated currents) using Claude API with confidence scoring. The designer reviews and commits the extracted data.

Tier 3 — Platform and Ecosystem	<i>Year 2+</i>
■ Multi-Engineer Collaboration	Multiple engineers work on the same design simultaneously. Digital sign-off at each chapter with initials and timestamp embedded in the report. Conflict resolution when two engineers modify the same section. Audit trail for regulatory submissions (IEC, UL, medical).
■ Manufacturing Output Packages	Generate: winding instructions for the inductor manufacturer (layer-by-layer, wire lengths, termination instructions), PCB footprint exports for the control circuit (Altium, KiCad), BOM in supplier-specific formats (Digi-Key, Mouser, Arrow).
■ SPICE Integration	Python generates a SPICE netlist from the DesignState. The designer runs it in LTSpice or SIMetrix and imports the simulation waveforms back into the report. The simulation figures show measured vs calculated comparison automatically.
■ Measured Data Integration	Import measured component values: LCR meter for inductor (L vs I curve), VNA for filter insertion loss, thermal camera for temperature map. The report automatically adds a "Measured vs Calculated" section to each chapter, showing how well the model predicts the real hardware.
■ AI-Powered Design Intelligence (full)	The AI advisor graduates from asking questions to suggesting fixes: "Replacing the 0059392A2 core with the 0059071A2 reduces ΔT by 6°C and improves saturation margin from $2.3\times$ to $2.8\times$ at a 12% cost increase." The designer chooses whether to accept the suggestion.

■ Regulatory Compliance Automation	Map every design parameter to the applicable regulatory requirement. Generate a compliance declaration: "This design meets IEC 61000-3-2 Class A at all 9 operating points per the harmonic analysis in Chapter 1." Flag any requirement that is not fully addressed by the current chapters.
---	--

Part 6 — Summary: What to Implement When

Priority	What	Why Now	Tier
1 — Critical (before anything else)	Canonical DesignState schema (DesignState.ts + schemas.py + DESIGN_STATE.md)	This is the foundation. Without it, every new agent and every new chapter adds another ad-hoc integration point. Doing it first costs 2 days. Retrofitting it later costs 2 weeks.	Tier 1
2 — High (current sprint)	Documentation Agent skill file Apply standards to Chapters 1–6	The documentation work is well-defined and agreed. The agent needs the DesignState schema to know what data is available.	Tier 1
3 — High (current sprint)	Chapters 7–9 calculation modules (stubs — interfaces only)	Define the TypeScript interfaces and Pydantic models for s7, s8, s9 as null-initialised sections. This costs 1 hour now and prevents a schema breaking change later.	Tier 1
4 — Medium (next sprint)	Full calculation and report for Chapter 7 (Power Semiconductors)	Follows the 12-step checklist from the Future Expansion Plan PDF. With the DesignState schema in place, this takes days not weeks.	Tier 1
5 — Future	AI Design Advisor Monte Carlo Analysis Design Optimisation	These require the Canonical DesignState to be stable. Start only after Chapters 1–9 are complete.	Tier 2
6 — Long-term	SPICE integration, measured data import, collaboration, compliance automation	These represent the full platform vision. Require stable architecture foundations from Tiers 1 and 2.	Tier 3

The most important action before implementation begins: Define the Canonical DesignState schema. Two days of work now eliminates months of integration pain later. Every other item on this roadmap becomes straightforward once the shared state contract is in place.

Generated by PFC AI Design Agent documentation planning session · June 2026 · Output:

C:\Users\toral\OneDrive\Desktop\pfc_ai_agent_v2\specs\PFC_Global_State_and_Next_Level.pdf